Robot Perception and Learning

Zero-shot RL

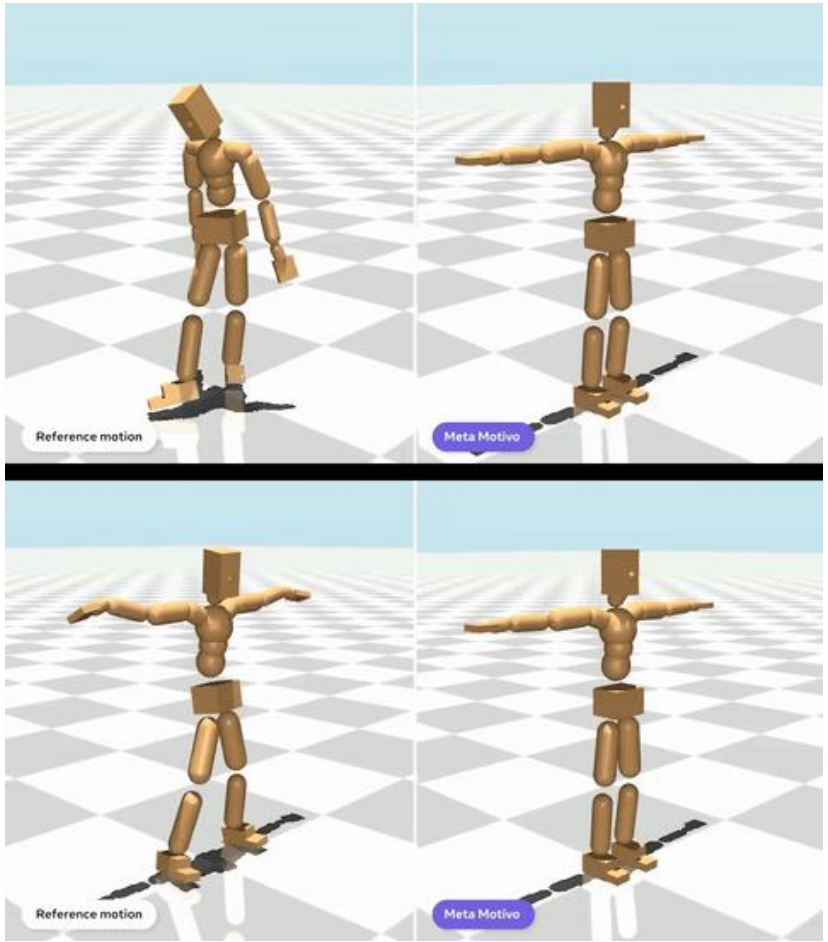
Tsung-Wei Ke

Fall 2026

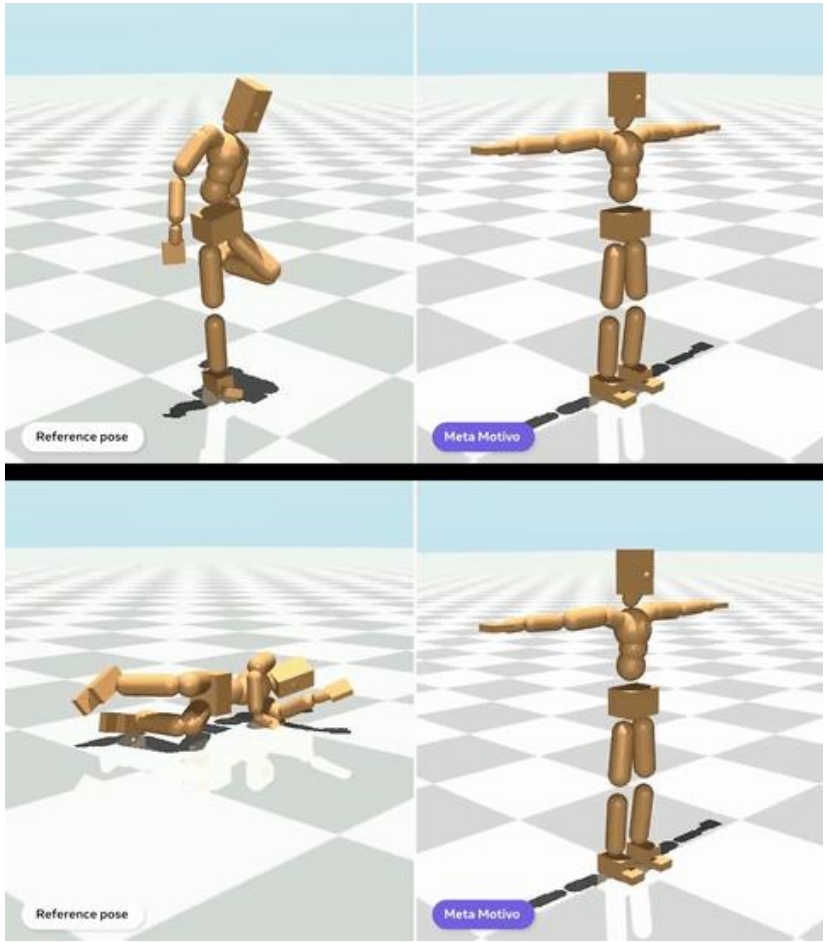


Problem: Can We Train a RL Policy that Supports Every Possible Task without any Reward?

Tracking reference motion

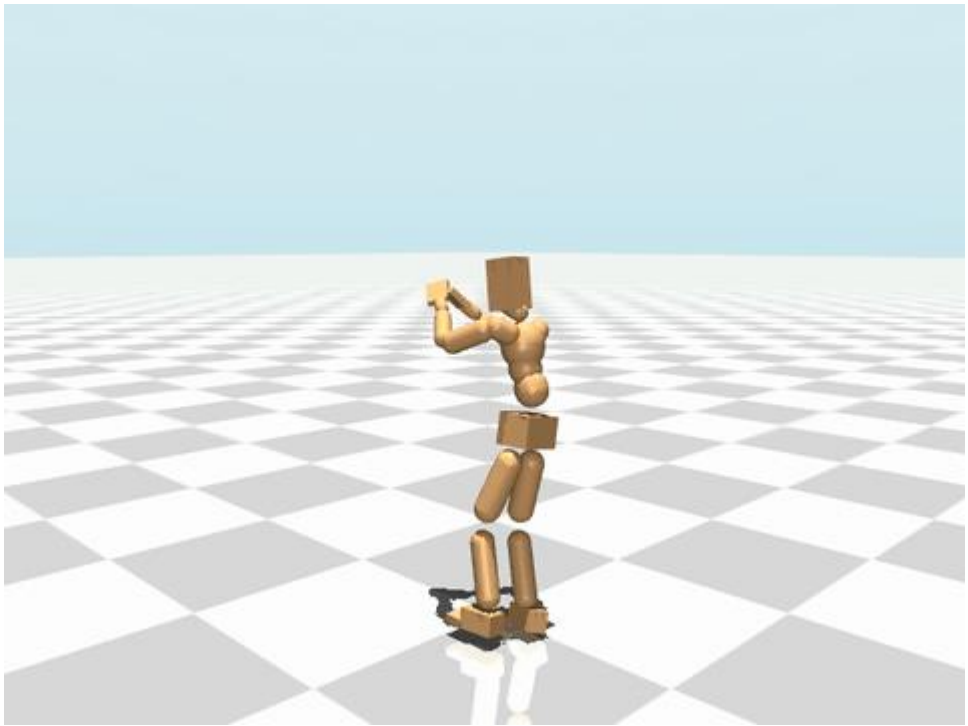


Reaching a goal pose



Problem: Can We Train a RL Policy that Supports Every Possible Task without any Reward?





Basic Locomotion

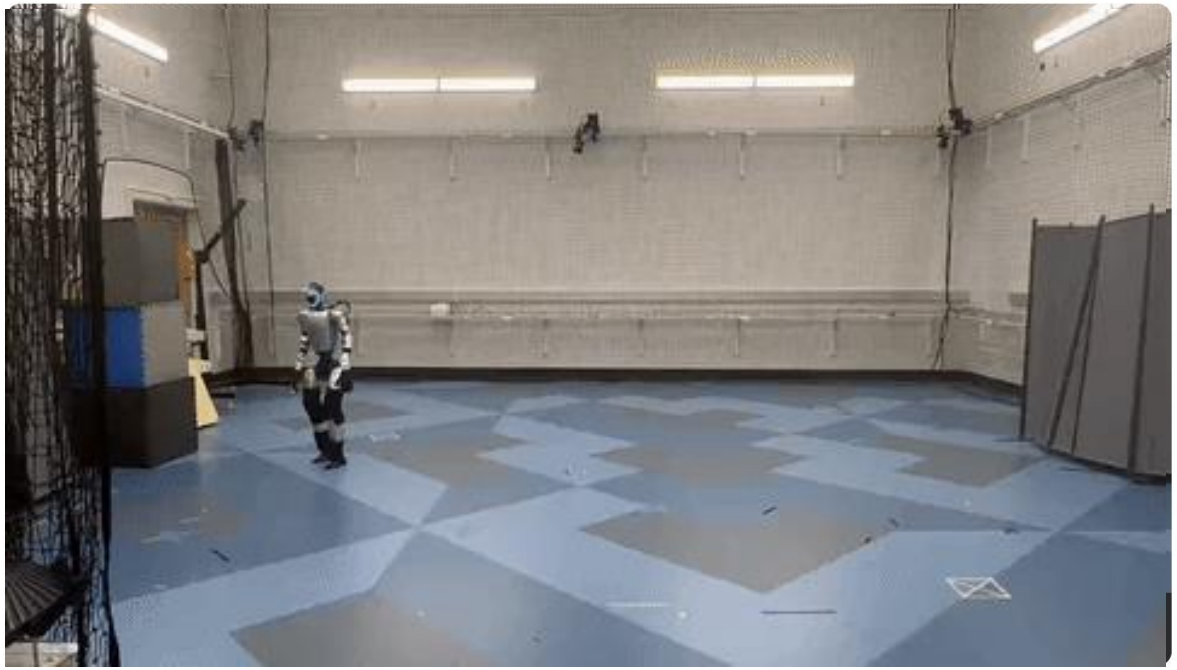
▶ Auto Play

We enable the robot to perform basic locomotion tasks including standing still, walking forward/backward/sideways, turning left/right.

- **Stand Still**
- **Go Forward**
- **Strafe Left**
- **Go Backward**
- **Strafe Right**
- **Turn Anticlockwise**
- **Turn Clockwise**

Forward walking at 0.7m/s

$$R = (\text{head_height} = 1.2\text{m}) \wedge (\text{base_vel_forward} = 0.7\text{m/s})$$



Basic Locomotion

▶ Auto Play

We enable the robot to perform basic locomotion tasks including standing still, walking forward/backward/sideways, turning left/right.

- **Stand Still**
- **Go Forward**
- **Strafe Left**
- **Go Backward**
- **Strafe Right**
- **Turn Anticlockwise**
- **Turn Clockwise**

Anticlockwise turning at 5.0 rad/s

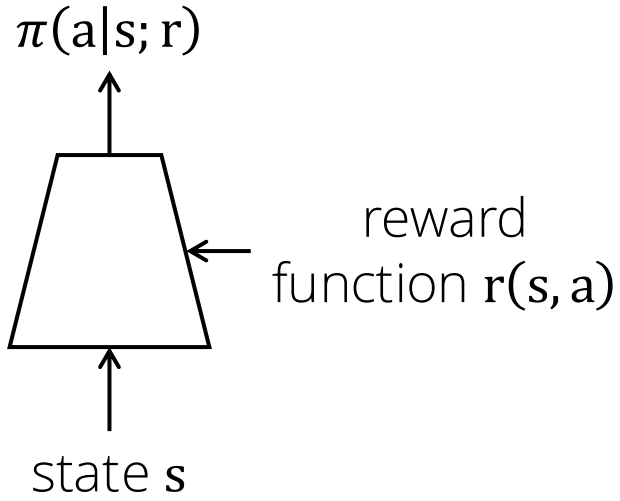
$$R = (\text{base_height} > 0.5\text{m}) \wedge (\text{base_ang_vel_z} = 5.0\text{rad/s})$$



Unsupervised Zero-Shot RL

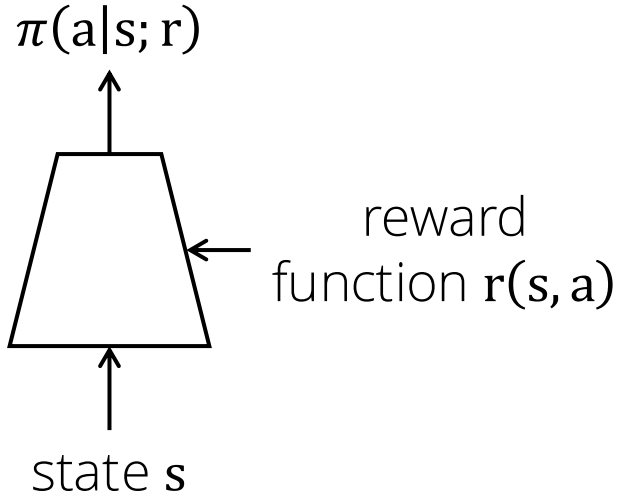
- Unsupervised zero-shot RL aims to offer a more general behavior learning framework than typical task-specific RL
  - ➢ During training, it learns a repertoire of policies without any task-specific reward functions
  - ➢ At test time, given a downstream reward function, it can immediately retrieve a corresponding policy without additional policy learning or planning

How to Retrieve the Optimal Policy for Any Reward?



- Idea: Learn a policy conditioned on reward functions
- During training, the policy learns on every random reward function
- At test time, the policy conditions on the input reward function

How to Retrieve the Optimal Policy for Any Reward?

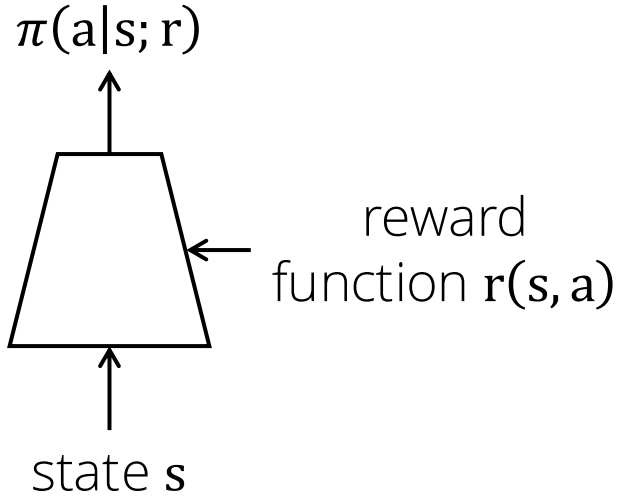


- Idea: Learn a policy conditioned on reward functions
- During training, the policy learns on every random reward function
- At test time, the policy conditions on the input reward function

How to learn such policies? Q-learning / Actor-critic learning!

$$\pi(a|s;r) = \arg\max_a Q(s,a;r)$$

How to Retrieve the Optimal Policy for Any Reward?



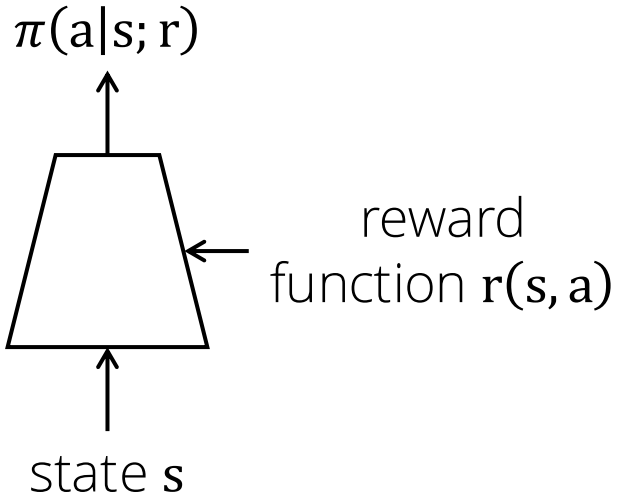
- Idea: Learn a policy conditioned on reward functions
- During training, the policy learns on every random reward function
- At test time, the policy conditions on the input reward function

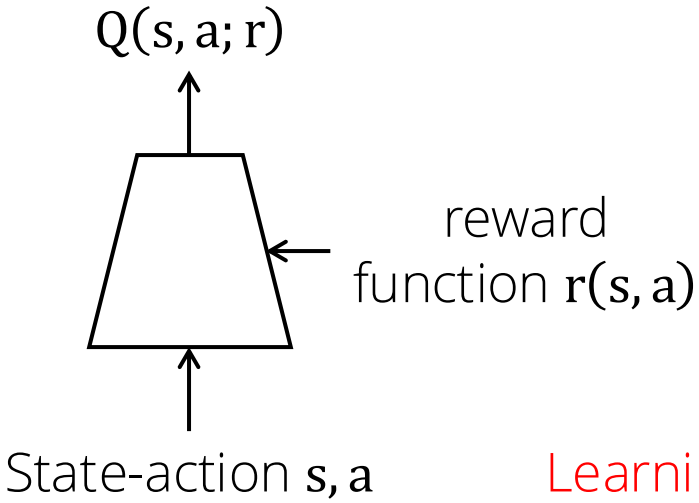
How to learn such policies? Q-learning / Actor-critic learning!

$$\pi(a|s;r) = \arg\max_a Q(s,a;r)$$

How to estimate $Q(s,a;r)$ for any reward?

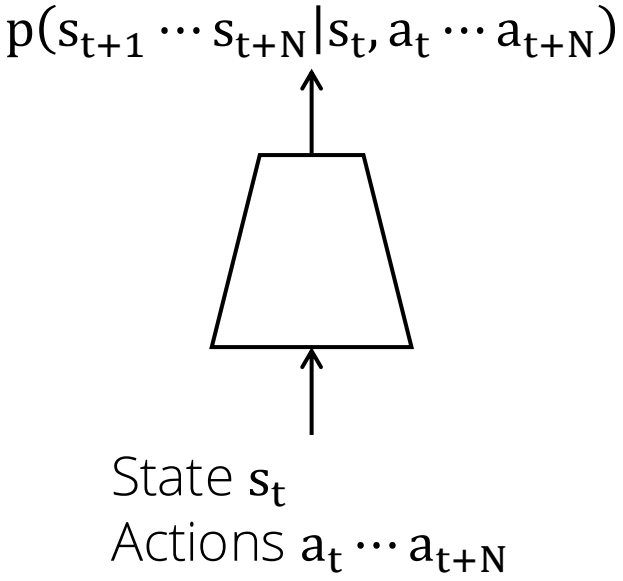
How to Learn the Optimal Policy for Any Reward?





Learning value function is hard, and learning values for any reward function is even harder

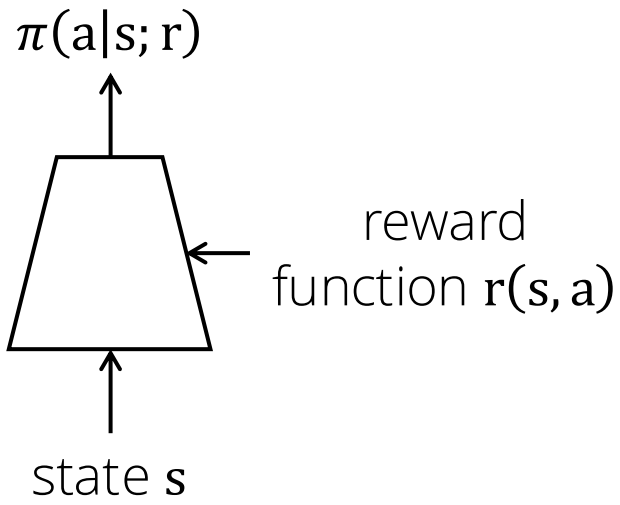
Learning Dynamics Models to Estimate Values for Any Rewards



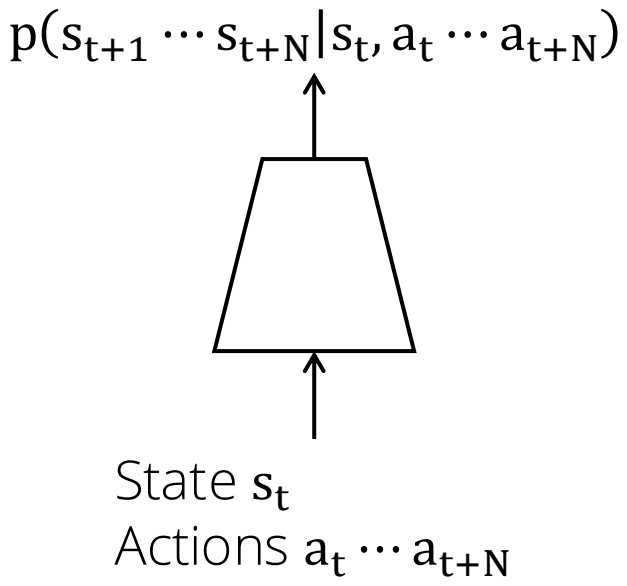
Value estimation:

$$Q(s, a; r) = r(s_t, a_t) + \mathop{E}_{s_{t+1} \cdots s_{t+N}} \left[\Sigma_{i=t+1}^{t+N} \gamma^{i+1} r(s_i, a_i) \right]$$

The Two Core Components of Zero-Shot RL



Reward-conditioned policy

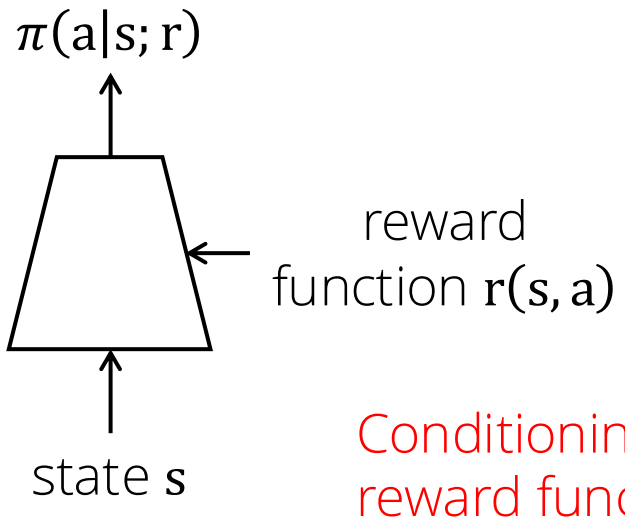


Dynamics model

Content

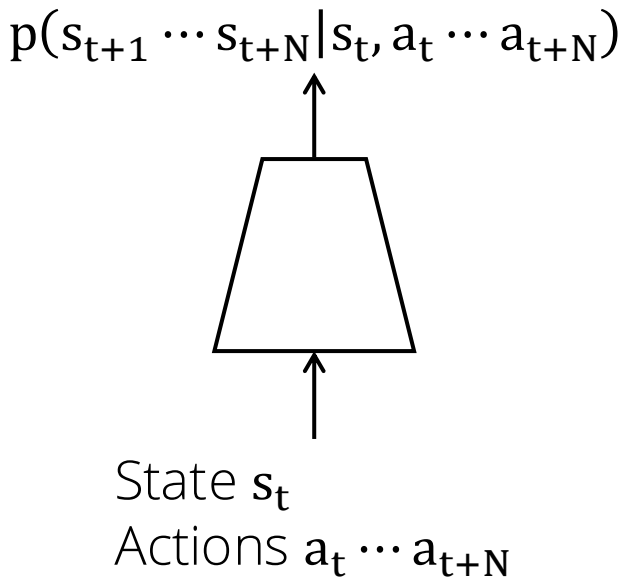
- Formulation of zero-shot RL
- Assumption of linear reward functions
- Re-formulation of dynamics models

The Two Core Components of Zero-Shot RL



Conditioning on non-linear reward functions is hard…

Reward-conditioned policy



Dynamics model

Linear Reward Functions

- Let $\phi(\mathbf{s})$ be the state encodings
- Assume all reward functions are linear and they live in the span of the learned features ϕ

$$\mathbf{r}(\mathbf{s}, \mathbf{a}) = \phi(\mathbf{s})^{\mathsf{T}}\mathbf{z}$$

In other words, we can represent a task specified by the reward function $\mathbf{r}(\mathbf{s}, \mathbf{a})$ with a vector $\mathbf{z}$

Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$\mathrm{m}_{\gamma}^{\pi}(\mathrm{x}|\mathrm{s},\mathrm{a}) = (1-\gamma)\Sigma_{\mathrm{k}=0}^{\infty}\,\gamma^{\mathrm{k}}\,\Pr(\mathrm{s}_{\mathrm{k}}=\mathrm{x}|\mathrm{s},\mathrm{a};\pi)$$

The probability of visiting state $\mathbf{x}$ at time step $\mathbf{k}$

$$\underbrace{p_\theta(\mathbf{s}_1,\mathbf{a}_1,\ldots,\mathbf{s}_T,\mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1)\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)$$

Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$\mathrm{m}_{\gamma}^{\pi}(\mathrm{x}|\mathrm{s},\mathrm{a}) = (1-\gamma)\Sigma_{\mathrm{k}=0}^{\infty}\,\gamma^{\mathrm{k}}\,\Pr(\mathrm{s}_{\mathrm{k}}=\mathrm{x}|\mathrm{s},\mathrm{a};\pi)$$

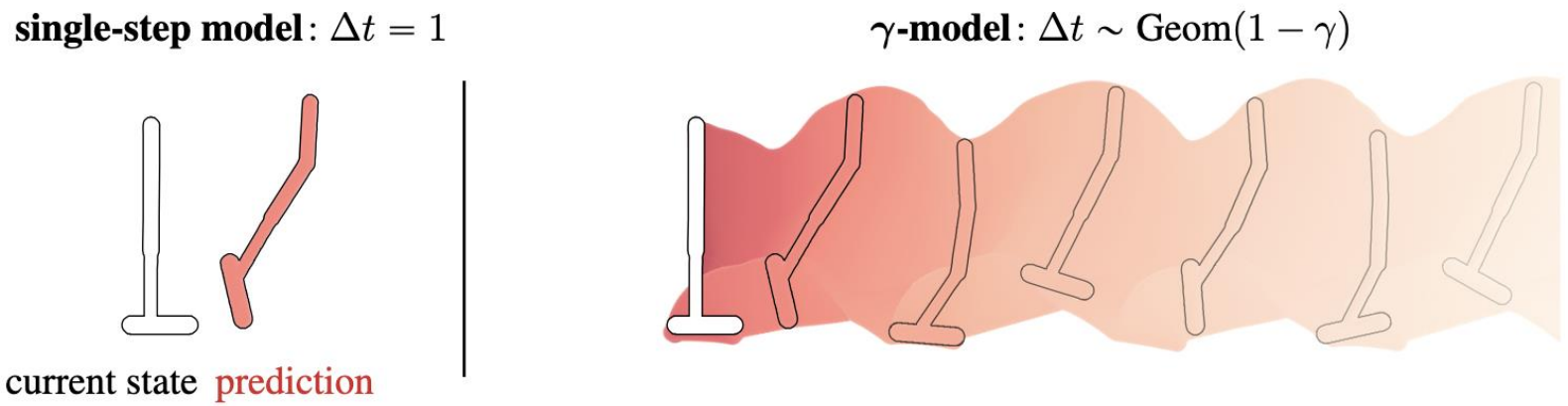
The probability of visiting state $\mathbf{x}$ at time step $\mathbf{k}$

$$\underbrace{p_\theta(\mathbf{s}_1,\mathbf{a}_1,\ldots,\mathbf{s}_T,\mathbf{a}_T)}_{p_\theta(\tau)} = p(\mathbf{s}_1)\prod_{t=1}^{T}\pi_\theta(\mathbf{a}_t|\mathbf{s}_t)p(\mathbf{s}_{t+1}|\mathbf{s}_t,\mathbf{a}_t)$$

Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$\mathrm{m}_{\gamma}^{\pi}(\mathrm{x}|\mathrm{s},\mathrm{a}) = (1-\gamma)\Sigma_{\mathrm{k}=0}^{\infty}\,\gamma^{\mathrm{k}}\,\Pr(\mathrm{s}_{\mathrm{k}} = \mathrm{x}|\mathrm{s},\mathrm{a};\pi)$$



Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$m_{\gamma}^{\pi}(x|s,a) = (1-\gamma)\Sigma_{k=0}^{\infty}\gamma^{k}\Pr(s_k = x|s,a;\pi)$$

- Value estimation:

$$\boxed{Q(s_t,a_t;r,\pi)} = \mathop{E}_{s_t\cdots s_{t+N}}\left[\Sigma_{i=t}^{t+N}\gamma^{i+1}r(s_i,a_i)\right] = \mathop{E}_{\substack{s^+\sim m_{\gamma}^{\pi}(\cdot|s_t,a_t)\\ a^+\sim\pi(\cdot|s^+)}}[r(s^+,a^+)]$$

A policy π's Q-value estimation under reward function r

Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$m_\gamma^\pi(x|s,a) = (1-\gamma)\Sigma_{k=0}^{\infty}\gamma^k \Pr(s_k = x|s,a;\pi)$$

- Value estimation:

$$Q(s_t,a_t;r,\pi) = \mathop{E}_{s_t\cdots s_{t+N}}\left[\Sigma_{i=t}^{t+N}\gamma^{i+1}r(s_i,a_i)\right] = \mathop{E}_{\substack{s^+\sim m_\gamma^\pi(\cdot|s_t,a_t)\\ a^+\sim\pi(\cdot|s^+)}}[r(s^+,a^+)]$$

$$= \boxed{M_\gamma^\pi}[r(s_1) \quad \cdots \quad r(s_N)]$$

A matrix form of occupancy measure

Estimate Values with vector $\mathbf{z}$

- Occupancy measure: the discounted distribution over future states visited by a policy from a state-action pair

$$m_\gamma^\pi(x|s,a) = (1-\gamma)\Sigma_{k=0}^{\infty}\,\gamma^k\,\Pr(s_k = x|s,a;\pi)$$

- Value estimation:

$$Q(s_t,a_t;r,\pi) = \mathop{E}_{s_t\cdots s_{t+N}}\left[\Sigma_{i=t}^{t+N}\,\gamma^{i+1}r(s_i,a_i)\right] = \mathop{E}_{\substack{s^+\sim m_\gamma^\pi(\cdot|s_t,a_t)\\ a^+\sim\pi(\cdot|s^+)}}[r(s^+,a^+)]$$

$$= M_\gamma^\pi[r(s_1) \quad \cdots \quad r(s_N)] = M_\gamma^\pi[\phi(s_1) \quad \cdots \quad \phi(s_N)]^T z = M_\gamma^\pi \Phi^T z$$

An Online Zero-Shot RL Algorithm

- Assume all reward functions are linear and they live in the span of the learned features ϕ

$$\mathrm{r}(\mathrm{s},\mathrm{a}) = \phi(\mathrm{s})^{\intercal}\mathrm{z}$$

- Pseudo learning algorithm:
  1. Sample random $\mathbf{z}$
  2. Collect trajectories with policy conditioned on $\mathbf{z}$, and store trajectories in the replay buffer
  3. Sample state-action pairs $(\mathbf{s},\mathbf{a})$ from the replay buffer, train policy conditioned on $\mathbf{z}$ to maximize the Q-value

$$\pi_{\mathrm{z}}(\mathrm{a}|\mathrm{s}) = \arg\max_{\mathrm{a}} \mathrm{Q}(\mathrm{s},\mathrm{a};\ \mathrm{z}) = \arg\max_{\mathrm{a}} \mathrm{M}_{\gamma}^{\pi_{\mathrm{z}}}(\cdot\,|\mathrm{s},\mathrm{a})\Phi^{\intercal}\mathrm{z}$$

An Online Zero-Shot RL Algorithm

- Assume all reward functions are linear and they live in the span of the learned features ϕ

$$\mathrm{r}(\mathrm{s},\mathrm{a}) = \phi(\mathrm{s})^{\intercal}\mathrm{z} \Rightarrow \mathrm{z} = \arg\max_{\mathrm{z}} \mathrm{E}_{\mathrm{s,a}}[\|\mathrm{r}(\mathrm{s},\mathrm{a}) - \phi(\mathrm{s})^{\intercal}\mathrm{z}\|_2^2]$$

- Pseudo learning algorithm:

Obtain $\mathbf{z}$ from the reward function during inference

1. Sample random $\mathbf{z}$
2. Collect trajectories with policy conditioned on $\mathbf{z}$, and store trajectories in the replay buffer
3. Sample state-action pairs $(\mathbf{s},\mathbf{a})$ from the replay buffer, train policy conditioned on $\mathbf{z}$ to maximize the Q-value

$$\pi_{\mathrm{z}}(\mathrm{a}|\mathrm{s}) = \arg\max_{\mathrm{a}} \mathrm{Q}(\mathrm{s},\mathrm{a};\ \mathrm{z}) = \arg\max_{\mathrm{a}} \mathrm{M}_{\gamma}^{\pi_{\mathrm{z}}}(\cdot\,|\mathrm{s},\mathrm{a})\Phi^{\intercal}\mathrm{z}$$

Learning One Representation to Optimize All Rewards

Ahmed Touati*
Mila, University of Montreal
ahmed.touati@umontreal.ca

Yann Ollivier
Facebook Artificial Intelligence Research
Paris
yol@fb.com

- A seminal zero-shot RL algorithm: forward-backward representations
- Low-rank approximation of occupancy matrix $\mathrm{M}_{\gamma}^{\pi_z}$

$$\mathrm{M}_{\gamma}^{\pi_z}(s^+|s,a) = \underbrace{\mathrm{F}(s,a;\ z)}^{\intercal}\underbrace{\mathrm{B}(s^+)}$$

Both are $\mathbf{d}$-dimensional latent vectors

Forward-backward (FB) Representation

- Low-rank approximation of occupancy matrix $\mathrm{M}_{\gamma}^{\pi_{\mathrm{z}}}$

$$\mathrm{M}_{\gamma}^{\pi_{\mathrm{z}}}(\mathrm{s}^{+}|\mathrm{s},\mathrm{a}) = \mathrm{F}(\mathrm{s},\mathrm{a};\ \mathrm{z})^{\intercal}\mathrm{B}(\mathrm{s}^{+})$$

- Assume the task embedding is linearly represented by reward functions

$$\mathrm{z} = \begin{bmatrix} | & | & | \\ \mathrm{B}(\mathrm{s}_1) & \cdots & \mathrm{B}(\mathrm{s}_{\mathrm{N}}) \\ | & | & | \end{bmatrix} \begin{bmatrix} \mathrm{r}(\mathrm{s}_1) \\ \vdots \\ \mathrm{r}(\mathrm{s}_{\mathrm{N}}) \end{bmatrix} = \mathrm{Br}$$

- Value estimation:

$$\mathrm{Q}(\mathrm{s},\mathrm{a};\mathrm{r},\pi) = \mathrm{Q}(\mathrm{s},\mathrm{a};\mathrm{z},\mathrm{z}') = \mathrm{M}_{\gamma}^{\pi_{\mathrm{z}'}}\mathrm{r} = \mathrm{F}(\mathrm{s},\mathrm{a};\ \mathrm{z}')^{\intercal}\mathrm{z}$$

Pseudo Algorithm of FB

- Low-rank approximation of occupancy matrix $\mathbf{M}_{\gamma}^{\pi_{\mathbf{z}}}$

$$\mathbf{M}_{\gamma}^{\pi_{\mathbf{z}}}(\mathbf{s}^{+}|\mathbf{s},\mathbf{a}) = \mathbf{F}(\mathbf{s},\mathbf{a};\ \mathbf{z})^{\mathsf{T}}\mathbf{B}(\mathbf{s}^{+})$$

- Assume the task embedding is linearly represented by reward functions

$$\mathbf{z} = \mathbf{B}\mathbf{r}$$

- Pseudo learning algorithm:
  1. Sample random $\mathbf{z}$
  2. Collect trajectories with policy conditioned on $\mathbf{z}$, and store trajectories in the replay buffer
  3. Sample state-action pairs $(\mathbf{s},\mathbf{a})$ from the replay buffer, train policy conditioned on $\mathbf{z}$ to maximize the Q-value

$$\pi_{\mathbf{z}}(\mathbf{a}|\mathbf{s}) = \arg\max_{\mathbf{a}} \mathbf{Q}(\mathbf{s},\mathbf{a};\ \mathbf{z}) = \arg\max_{\mathbf{a}} \mathbf{F}(\mathbf{s},\mathbf{a};\ \mathbf{z}')^{\mathsf{T}}\mathbf{z}$$

Pseudo Algorithm of FB

- Low-rank approximation of occupancy matrix $\mathbf{M}_{\gamma}^{\pi_{\mathbf{z}}}$

$$\mathbf{M}_{\gamma}^{\pi_{\mathbf{z}}}(\mathbf{s}^{+}|\mathbf{s},\mathbf{a}) = \mathbf{F}(\mathbf{s},\mathbf{a};\ \mathbf{z})^{\mathsf{T}}\mathbf{B}(\mathbf{s}^{+})$$

- Assume the task embedding is linearly represented by reward functions

$$\mathbf{z} = \mathbf{B}\mathbf{r}$$

- Pseudo learning algorithm:

Unfortunately unsupervised zero-shot RL doesn't work well. How about introduce human motion prior?

1. Sample random $\mathbf{z}$
2. Collect trajectories with policy conditioned on $\mathbf{z}$, and store trajectories in the replay buffer
3. Sample state-action pairs $(\mathbf{s},\mathbf{a})$ from the replay buffer, train policy conditioned on $\mathbf{z}$ to maximize the Q-value

$$\pi_{\mathbf{z}}(\mathbf{a}|\mathbf{s}) = \arg\max_{\mathbf{a}} \mathbf{Q}(\mathbf{s},\mathbf{a};\ \mathbf{z}) = \arg\max_{\mathbf{a}} \mathbf{F}(\mathbf{s},\mathbf{a};\ \mathbf{z}')^{\mathsf{T}}\mathbf{z}$$

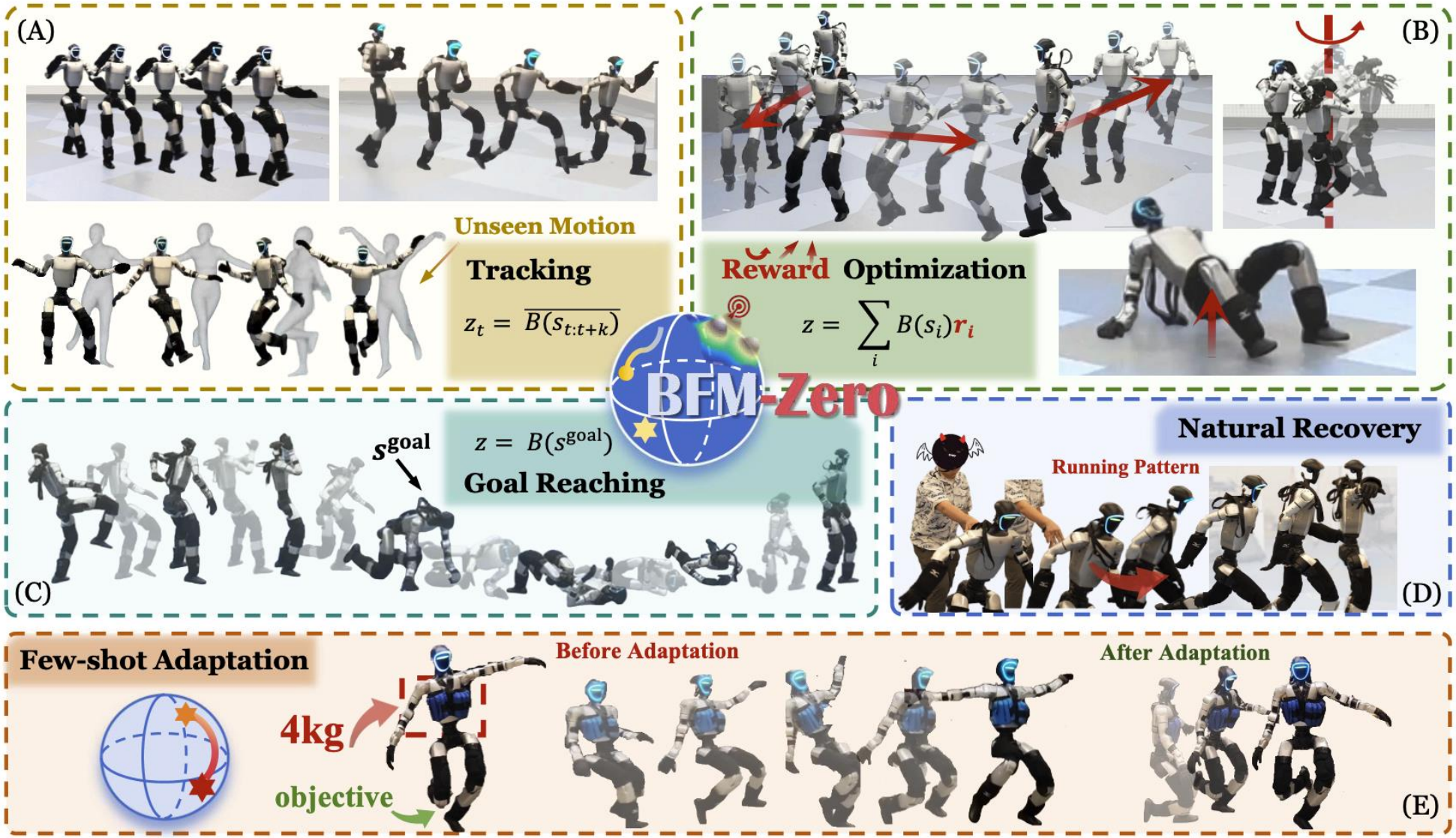
Zero-Shot Whole-Body Humanoid Control via Behavioral Foundation Models

Andrea Tirinzoni[1,*], **Ahmed Touati**[1,*], **Jesse Farebrother**[2,+], **Mateusz Guzek**[1], **Anssi Kanervisto**[1], **Yingchen Xu**[1,3], **Alessandro Lazaric**[1,†], **Matteo Pirotta**[1,†]

[1]FAIR at Meta, [2]Mila, McGill University, [3]UCL

*Joint first author, +Work done at Meta, †Joint last author

- Unsupervised zero-shot RL based on forward-backward representation, with human motion prior


(A)
Unseen Motion
Tracking
$z_t = \overline{B(s_{t:t+k})}$
(B)
Reward Optimization
$z = \sum_i B(s_i) r_i$
BFM-Zero
s^{goal}
$z = B(s^{goal})$
Goal Reaching
(C)
Natural Recovery
Running Pattern
(D)
Few-shot Adaptation
4kg
objective
Before Adaptation
After Adaptation
(E)

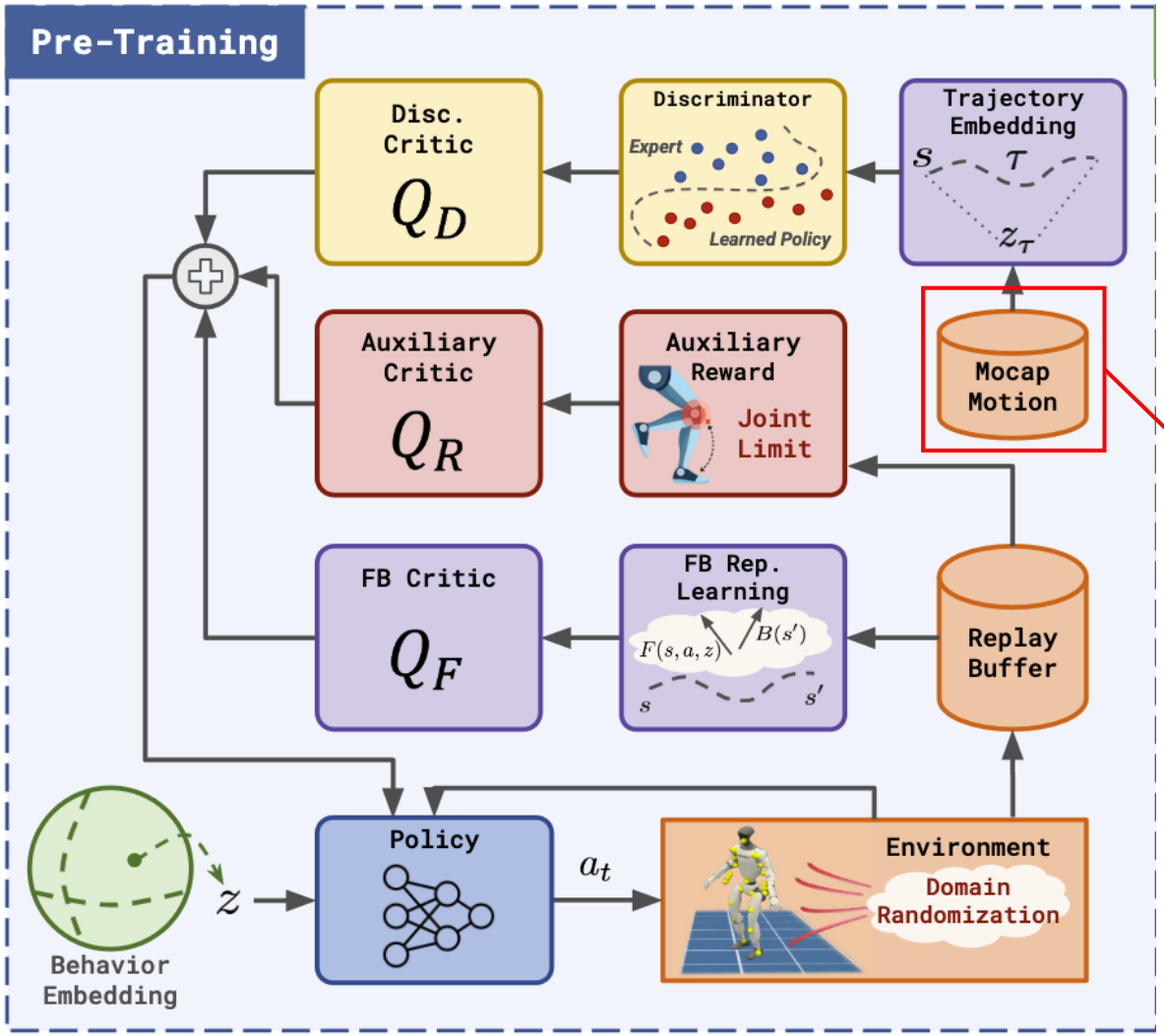
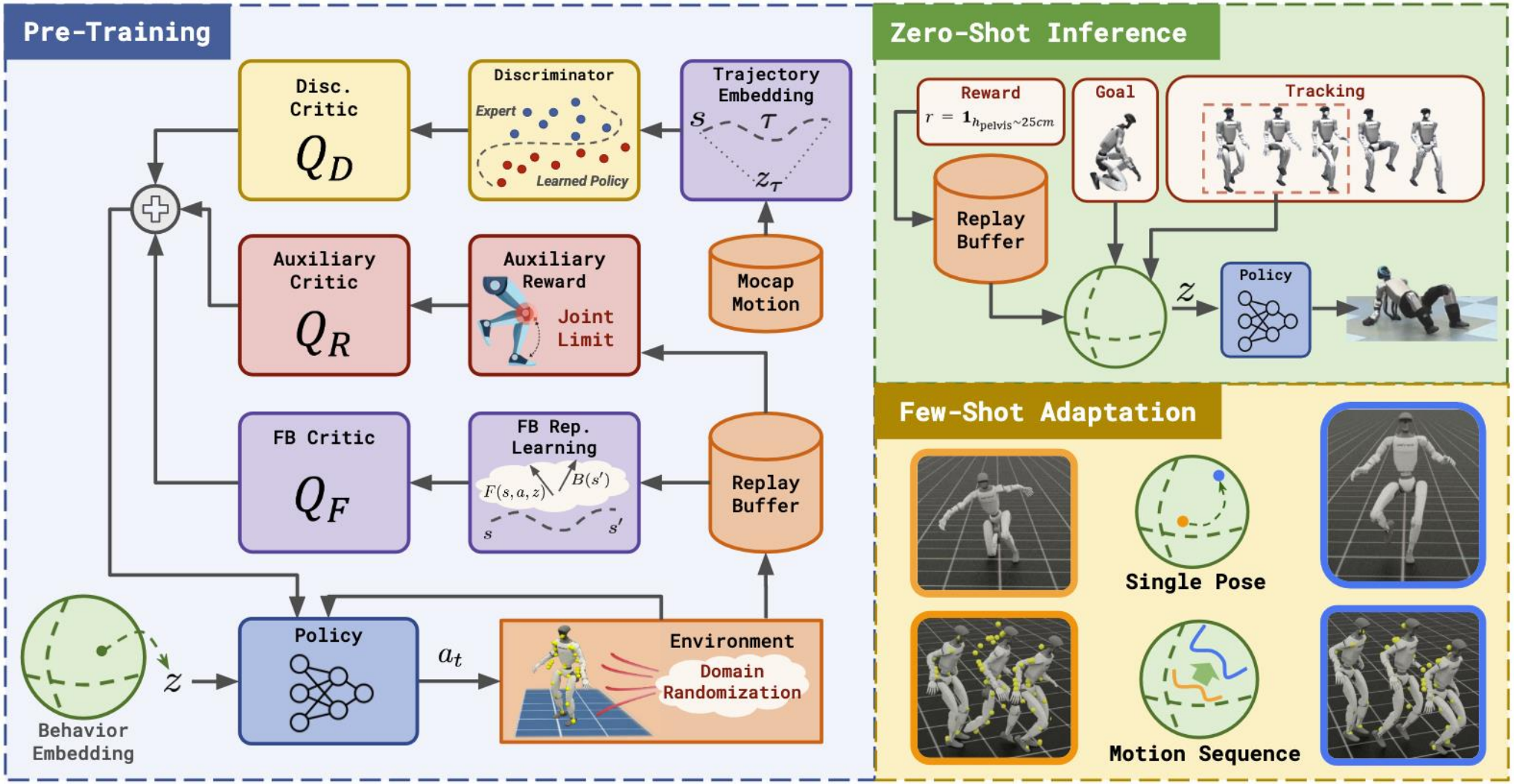




Image source: AMASS dataset


Pre-Training
Disc. Critic
Q_D
Discriminator
Expert
Learned Policy
Trajectory Embedding
s
τ
z_τ
Auxiliary Critic
Q_R
Auxiliary Reward
Joint Limit
Mocap Motion
FB Critic
Q_F
FB Rep. Learning
$F(s,a,z)$
$B(s')$
s
s'
Replay Buffer
Policy
a_t
Environment
Domain Randomization
z
Behavior Embedding
Zero-Shot Inference
Reward
$r = \mathbf{1}_{h_{\text{pelvis}} \sim 25cm}$
Goal
Tracking
Replay Buffer
z
Policy
Few-Shot Adaptation
Single Pose
Motion Sequence